

Linked List Operations in C

Implementation of Requested Functions

Below are the implementations for the three linked list operations you requested from your handwritten notes, separated into distinct code blocks.

Prerequisite Structure: Assume the following node structure is defined for all operations:

```
struct Node {  
    int data;  
    struct Node* next;  
};
```

1. How to add a new node at the beginning of the list

This function creates a new node, points its next pointer to the current head, and then updates the head to point to this new node.

```
void insertAtBeginning(struct Node** head_ref, int new_data) {  
    // 1. Allocate memory for the new node  
    struct Node* new_node = (struct Node*)malloc(sizeof(struct Node));  
  
    // 2. Assign data to the new node  
    new_node->data = new_data;  
  
    // 3. Make next of new node point to the current head  
    new_node->next = (*head_ref);  
  
    // 4. Move the head to point to the new node  
    (*head_ref) = new_node;  
}
```

2. Find the length of the list

This function traverses the list starting from the head, incrementing a counter for each node until it reaches the end (NULL).

```

int getLength(struct Node* head) {
    int count = 0; // Initialize count to 0
    struct Node* current = head; // Start at the head of the list

    // Traverse the list until the end is reached
    while (current != NULL) {
        count++; // Increment the counter
        current = current->next; // Move pointer to the next node
    }

    return count; // Return the total number of nodes
}

```

3. Display the data

This function walks through the linked list, printing the integer data stored in each node, formatted sequentially.

```

void displayList(struct Node* head) {
    struct Node* current = head; // Start at the head

    printf("Linked List Data: ");

    // Loop through the list
    while (current != NULL) {
        printf("%d -> ", current->data); // Print the data
        current = current->next; // Move to the next node
    }
    printf("NULL\n"); // Indicate the end of the list
}

```